JETBRAINS | Kotlin

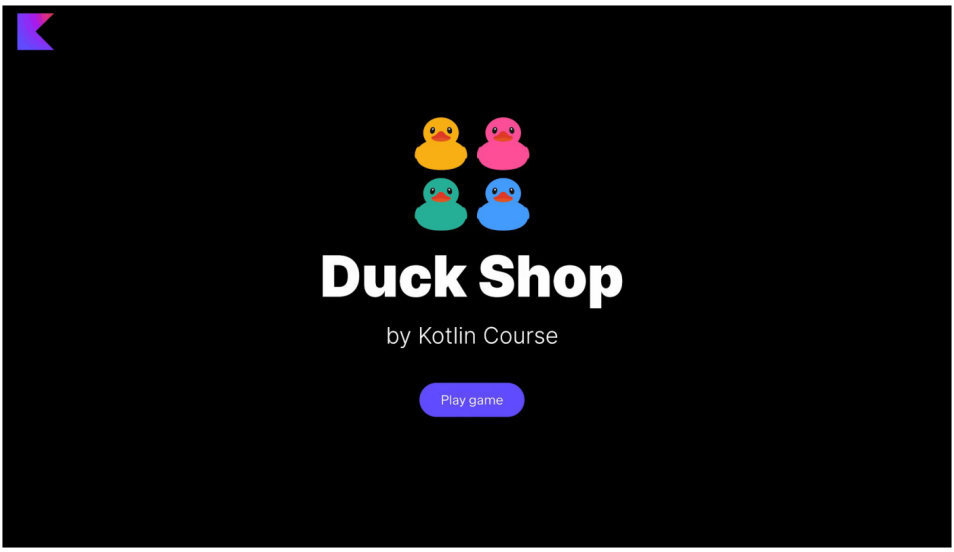
Backend Development with Kotlin

kotlinglang.org/education



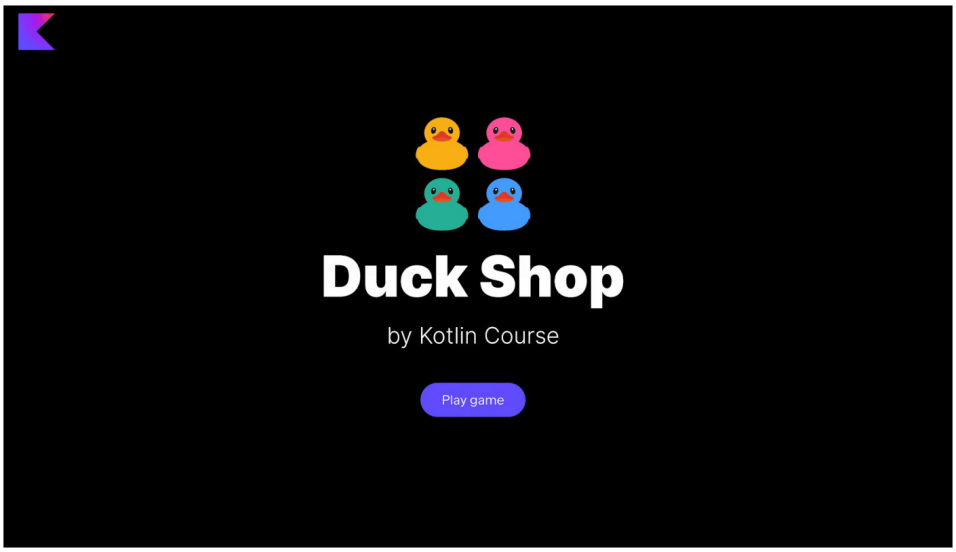
From HTTP to Spring and Ktor: Using Duck Shop as an Example

Frontend (fixed) ->
Server in Kotlin
(focus of the class)



From HTTP to Spring and Ktor: Using Duck Shop as an Example

Frontend (fixed) ->
Server in Kotlin
(focus of the class)



One project:

Spring vs Ktor

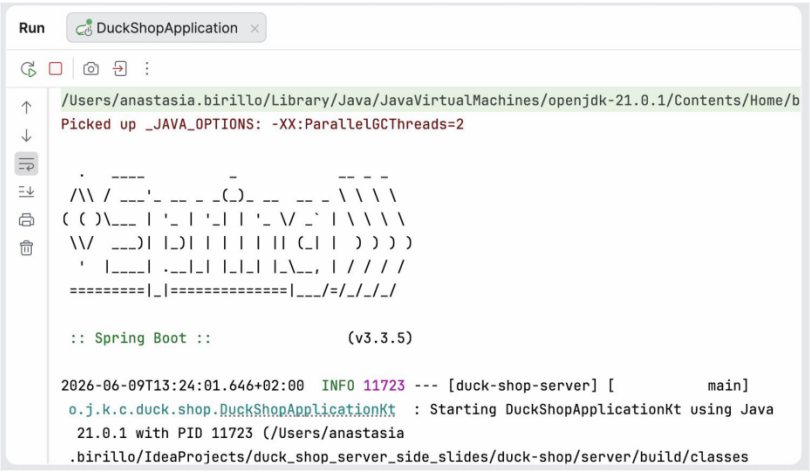
All main components

What is a backend?

- **The server side of an app** — users don't interact with it directly
- Owns **business logic, data storage, and security**
- **Serves data to frontends** (web, mobile) and talks to databases & other services
- Runs on a server, usually exposed as a set of **APIs**

Backend vs Frontend

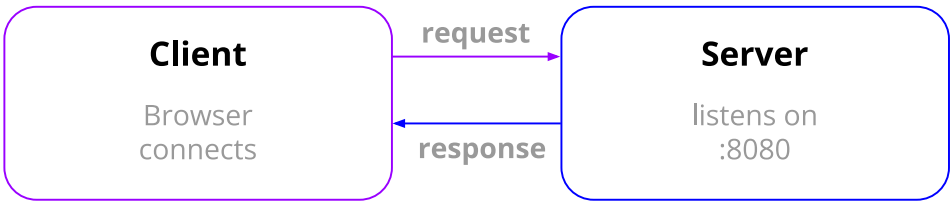


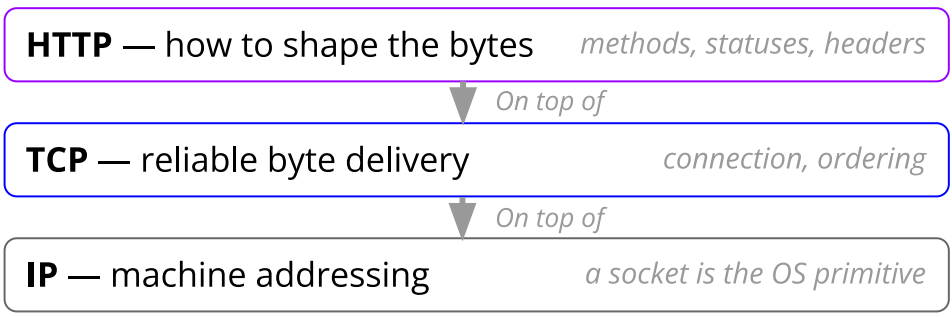


The frontend displays the shop, while the backend stores all the data and handles all the operations

From sockets to HTTP

- **IP address** — who (a machine's address on the network, e.g. `127.0.0.1`)
- **Port** — which channel (a channel on the machine, 0–65535. *Our server listens on `8080`)*
- **Connection** — a pair of endpoints (source `IP:port`) ↔ (destination `IP:port`)
- **Client ↔ server** (the server listens and waits; the client connects. *In our project the browser is the client*)
- **localhost** — your own machine (this is how we test without a network)





TCP in action

The rawest level: open a socket by hand and send bytes over a connection

```
   val server = ServerSocket(9999)     // 
listen on a port
   println("Listening on 9999...")
   val client = 
server.accept()       // block until a client 
connects

   val writer = 
PrintWriter(client.getOutputStream(), true)
   writer.println("Hello from a raw Kotlin 
socket!")
   client.close();
   server.close()
```

Tip: run the snippet and connect as a client: `nc localhost 9999`

‼️ One client, one message, everything by hand — no routes, no data format, no concurrent requests.

Same transport but structured: HTTP

The **same** TCP connection — but with an agreed format on top. Try it out with Duck Shop:

```
$ curl -v http://localhost:8080/api/ducks

> GET /api/ducks HTTP/1.1                ← request
line: method · path · version
> Host: localhost:8080                   ← request
headers
> Accept: */*
>
< HTTP/1.1 200 OK                        ← status
line
< Content-Type: application/json
← response headers
<
[{"name":"Piter","hasKotlinAttribute":false}, ...
← body (JSON)
```

‼️ The transport is still TCP — but now there's a **method**, a **path**, **headers**, a **status code**, and a **typed body**. HTTP handles routing, formats, and concurrency for us

Anatomy of an HTTP* message

Request

```
GET /api/ducks HTTP/1.1        method · path · version
Host: localhost:8080           headers
Accept: */*

(optional body — e.g. on POST)
```

Response

```
HTTP/1.1 200 OK                  status line
Content-Type: application/json   headers

[{"name":"Piter", ...}]           body
```

*HTTP/1.x is a text-based protocol; it carries both text and binary payloads

Web APIs

REST
Resources + HTTP methods. Simple, widely used, cacheable.
good for: most web / mobile backends

GraphQL
One endpoint; the client queries exactly the fields it needs.
good for: rich, varied clients

gRPC + Protobuf
Binary, contract-first, low overhead and latency.
good for: service-to-service / microservices

WebSockets
Persistent, bidirectional connection for live data.
good for: real-time (chat, live updates)

*SOAP (XML, heavyweight) — still around in legacy systems, rarely chosen for new APIs

REST

A set of conventions — follow them and clients become easy to reason about, cache, and scale

- **Resource-based** — URLs identify resources
- **HTTP methods** — GET / POST / PUT / PATCH / DELETE
- **Stateless** — the server keeps no per-client session between requests
- **Uniform interface** — consistent, predictable interaction patterns
- **Client–server separation** — both sides evolve independently

HTTP methods in REST

The method tells the server (and caches and proxies) your intent — before it reads the body

GET

Read, **safe*** (no side effects)

`GET /api/ducks`

POST

Create / trigger an action

`POST /api/ducks`

PUT

Full replace of a resource (idempotent*)

`PUT /api/ducks?mode=List`

PATCH

Partial update of a resource

`PATCH /api/ducks {order}`

DELETE Remove a resource (idempotent*)

`DELETE /api/ducks/{index}`

*Safe == doesn't change state; idempotent == repeating it has the same effect

Resources & URLs

Endpoints name resources (nouns); query params refine them

- **Collection**

GET `/api/ducks`

- **Individual**

GET `/api/ducks/{index}`

- **Filtering a collection**

`DELETE /api/ducks?hasKotlin=false`

- **State as a property**

`PATCH /api/ducks {order:"price"}`

‼️ Without a database, the collection of ducks is stored on the server. This is fine for a toy project, but it won't work for a real one.

HTTP status codes

The first thing a client checks — the status code, before parsing any body

2xx

`Success`

200 OK; 201 Created; 204 No Content

3xx

`Redirection`

301 Moved Permanently; 304 Not Modified

4xx

`Client error`

400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found

5xx

`Server error`

500 Internal Server Error; 503 Service Unavailable

```
$ curl -v http://localhost:8080/api/
ducks  # 200
$ curl -v http://localhost:8080/
unknown    # 404
```

Why a framework — and what it gives you

A raw socket lacks routes, formats, and concurrency — framework provides scaffolding:

- **HTTP server & routing**
- **Request/response & serialization**
- **Validation & error handling**
- **Database integration**
- **Authentication & authorization**
- **Testing support**
- **Logging & metrics**

Spring & Spring Boot*

We build on Spring — and let Spring Boot remove the configuration boilerplate

Spring Boot
"Just run" on top of Spring

- Auto-configuration
- **Starters** — curated dependency bundles
- **Embedded server** (Tomcat) — no app server to install
- Stand-alone, runnable jar

Spring
The application framework

- IoC / Dependency Injection container
- Web (Spring MVC)
- Data access
- Security
- Testing

*Building web applications with Spring Boot and Kotlin: link

Setting up & building duck-shop

Generate at start.spring.io (`Gradle Kotlin DSL; Kotlin; JVM 21`):

```
plugins {
   kotlin("jvm") version "2.2.20"
   kotlin("plugin.spring") version "2.2.20"
   id("org.springframework.boot") version "4.0.0"
   id("io.spring.dependency-management") version
"1.1.7"
}

dependencies {

implementation("org.springframework.boot:spring-
boot-starter-web")

implementation("org.springframework.boot:spring-
boot-starter-security")

implementation("org.springframework.boot:spring-
boot-starter-data-jpa")
   // + actuator, validation, jackson-module-
kotlin …
}

java { toolchain { languageVersion =
JavaLanguageVersion.of(21) } }
```

‼️ Our build also auto-builds the React frontend into the jar — **one full-stack artifact** serving both API and UI

The artifact & the runtime

One self-contained process — no external application server

```
$ ./gradlew bootJar
$ java -jar build/libs/duck-shop-server-0.0.1-SNAPSHOT.jar
… Tomcat started on port 8080 (http)
… Started DuckShopApplicationKt in 0.8 seconds
```

The artifact & the runtime

- **Fat jar** — your code + all dependencies + embedded Tomcat + the React build
- **Embedded server** — the app is the server, listens on `:8080`, nothing to deploy into
- `./gradlew bootRun` for development; `java -jar` in production; optional `Dockerfile`

Give it a try yourself (and don't forget to start the server):

```
$ curl http://localhost:8080/actuator/health
$ open http://localhost:8080
$ curl http://localhost:8080/api/ducks

# {"status":"UP"}
# the duck shop UI
# JSON
```

The first endpoint

Two **annotations** turn a class into a running web app

```
@SpringBootApplication
class DuckShopApplication

fun main(args: Array<String>) {
runApplication<DuckShopApplication>(*args) }

@RestController
class HelloController {
   @GetMapping("/hello")
   fun hello() = "Hello from Spring!"
}
```

`@SpringBootApplication` bootstraps everything

`@RestController` + `@GetMapping` map a function to an HTTP route — Spring serves it on the

embedded server, no wiring by hand

The IoC container & beans

Inversion of control: Spring builds and manages your objects (beans) — you declare, it owns the lifecycle

- The container **instantiates**, **configures** and **wires** beans at startup
- `@Component` (and its specializations) mark a class as a bean*
- **Component scanning** discovers them automatically — no manual registration
- You never create your controller or service — Spring does, once, and injects what they need

*Less glue code, single instances, easy to test (The old XML bean config is gone — annotations only)

Dependency injection — constructor style

The idiomatic Kotlin/Spring way: dependencies are constructor parameters

```
@RestController
@RequestMapping("/api/ducks")
class DuckResource(private val shop:
DuckShopService) {

    // ...endpoints use `shop`
}
```

- Spring sees the constructor needs a `DuckShopService` bean and injects it — a single constructor needs no `@Autowired`
- Dependencies are explicit, immutable (`val`) and easy to fake in tests
- The service gets its own collaborators the same way

Component specialization

All are `@Component` beans — but the annotation documents the role (and adds layer behavior)

`@RestController`

`@Service`

`@Repository`

Web layer — maps HTTP to calls

`DuckResource (= @Controller + @ResponseBody)`

Business logic

`DuckShopService, GameModeService, GameActionFunctionsService`

Data access (adds exception translation) *`JPA repositories, will be explained later`*

GET /api/ducks

A real read endpoint — thin controller delegating to the service

```
@RestController
@RequestMapping("/api/ducks")
class DuckResource(private val shop:
DuckShopService) {

   @GetMapping
   fun getDucks(): List<JsDuck> = shop.all()
}
```

```
$ curl -v http://localhost:8080/api/
ducks  # [{"name":"Piter",…}]

# the UI renders them
```

‼️ `@GetMapping` on the class path → `GET /api/ducks` returns `List<JsDuck>`, Spring serializes to JSON

Path & query parameters

Read values from the URL; Spring binds and converts the types

```
@DeleteMapping("/{index}")
fun removeDuck(@PathVariable index: Int):
List<JsDuck> =
    shop.removeAt(index)

@PutMapping
fun replaceCollection(@RequestParam mode:
String): List<JsDuck> =
    shop.replaceWith(mode.toGameMode())
```

`DELETE /api/ducks/3` — a **path variable** identifies a resource segment `PUT /api/ducks?mode=List` — a **query param** refines the request. Spring converts `index` → `Int`

Serialization — domain vs DTO

Don't expose the entity straight to clients — map it to a DTO shaped for the API

```
// API DTO — what the client sees
data class JsDuck(val name: String, val
description: String?, val hasKotlinAttribute:
Boolean)

fun Duck.toJsDuck(mode: GameMode): JsDuck =
   JsDuck(displayName ?: name,
hasKotlinAttribute, getJsDescription(mode))
```

`jackson-module-kotlin` serializes Kotlin classes automatically (both ways) The DTO is shaped for the client — e.g. `description` is only set in `Map` mode — while the entity stays internal

@RequestBody

Bind a JSON request body to a Kotlin object

```
data class ReorderRequest(val order: String)

@PatchMapping
fun reorderDucks(@RequestBody request:
ReorderRequest): List<JsDuck> =
   shop.reorder(request.order.toOrder())
```

```
$ curl -u admin:admin -X PATCH -H
"Content-Type: application/json" \
       -d '{"order":"price"}' http://
localhost:8080/api/ducks

# or click “sort” in the UI
```

‼️ `PATCH /api/ducks with {"order":"price"}` → sort (also `"random"` / `"kotlin-first"`) One endpoint, three orderings

Error handling

One consistent error shape for the whole API.

```
data class ApiError(val status: Int, val
error: String, val message: String?)

@RestControllerAdvice
class ApiExceptionHandler {

@ExceptionHandler(ResponseStatusException::class)
→ ApiError

@ExceptionHandler(IllegalArgumentException::class
→ 400
}
```

`@RestControllerAdvice` centralizes exception→JSON mapping; controllers stay clean
`DELETE /api/ducks/999` → 404 & `PUT ?mode=Nope` → 400 — both as the same `ApiError` body

Databases & CRUD

So far the ducks lived in memory and vanished on restart — we need durable storage

- **Database** — data organized for fast search & retrieval
- **DBMS** — the software that manages a database
- **CRUD** — the four basic operations: **C**reate **R**ead **U**pdate **D**elete

Our duck operations map onto CRUD:

add → Create

list → Read

reorder → Update

remove/filter → Delete

SQL vs NoSQL

Two families — pick by the shape of your data

SQL (relational)
Tables, rows, columns. Structured, strong consistency, relationships.
H2; PostgreSQL; MySQL → our choice

NoSQL
Documents / key-value / wide-column. Flexible schema, easy horizontal scale.
MongoDB; Cassandra

‼️ Duck-shop is structured and relational — ducks have accessories → SQL. `H2` in dev, `PostgreSQL` in tests (see next)

Spring Data JPA

The layer that maps Kotlin objects to database rows — before we write any code

- **JPA** — the standard for mapping objects ↔ tables (ORM)
- **Hibernate** — the implementation Spring Boot uses by default
- **Spring Data JPA** — repository interfaces with almost no boilerplate

- Dependencies: `spring-boot-starter-data-jpa` + `kotlin("plugin.jpa")`

‼️ You write entities and repository interfaces — Spring & Hibernate generate the SQL

JPA entities — @Entity, @Id

Each entity maps to a table; `@Id` is the primary key

```
@Entity
class Accessory(
    val name: String = "",
    val price: Int = 0,
    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    val id: Long? = null,
)
```

The Kotlin JPA plugin generates the no-arg constructor Hibernate needs, so we keep *idiomatic* Kotlin

`@GeneratedValue` lets the database assign the id

Relationships — Duck ↔ Accessory

An accessory like “Hat” is shared by several ducks → **many-to-many**

```
@Entity
class Duck(
    val name: String = "", val displayName:
String? = null,
    val hasKotlinAttribute: Boolean = false,
    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "duck_accessories",
        joinColumns = [JoinColumn(name =
"duck_id")],
        inverseJoinColumns = [JoinColumn(name
= "accessory_id")],
    )
    val accessories: Set<Accessory> =
emptySet(),
    @Id @GeneratedValue(strategy =
GenerationType.IDENTITY) val id: Long? = null,
)
```

‼️ `@ManyToMany` → JPA creates a **join table** `duck_accessories`

Repositories — Spring Data

All are `@Component` beans — but the annotation documents the role (and adds layer behavior)

`@RestController`

Web layer — maps HTTP to calls

`DuckResource (= @Controller + @ResponseBody)`

`@Service`

Business logic

`DuckShopService, GameModeService, GameActionFunctionsService`

`@Repository`	Data access (adds exception translation) *JPA repositories, will be explained later*

Repositories — Spring Data

```
@Repository
interface DuckRepository :
JpaRepository<Duck, Long>

@Repository
interface ShopItemRepository :
JpaRepository<ShopItem, Long> {
   fun findAllByOrderByPositionAsc():
List<ShopItem>
}
```

Extend `JpaRepository` → `findAll` / `save` / `delete` / `count` for free

Derived queries come from the method name (`findAllByOrderByPositionAsc`) — no SQL written

Service + repositories

DB operations in transactions — and the web layer doesn't change at all

```
@Service
class DuckShopService(
    private val duckRepository: DuckRepository,
    private val shopItemRepository:
ShopItemRepository,
) {
    @Transactional
    fun addRandomDuck(): List<JsDuck>
{ /* pick from catalog, save a ShopItem */ }

    @Transactional(readOnly = true)
    fun all(): List<JsDuck> = /* read ordered
items → DTOs */
}
```

- `@Transactional` wraps each operation in a DB transaction
- `DuckResource` from earlier state is untouched — only storage moved
- A `CatalogSeeder` inserts the 10 ducks + accessories once, if the DB is empty

Config & persistence

Point Spring at a database — and the ducks finally survive a restart (`application.yml`)

```
spring:
 datasource:
   url: jdbc:h2:file:./data/
duckshop;DB_CLOSE_DELAY=-1  # file-based →
survives restart

 jpa:
   hibernate:
     ddl-auto: update

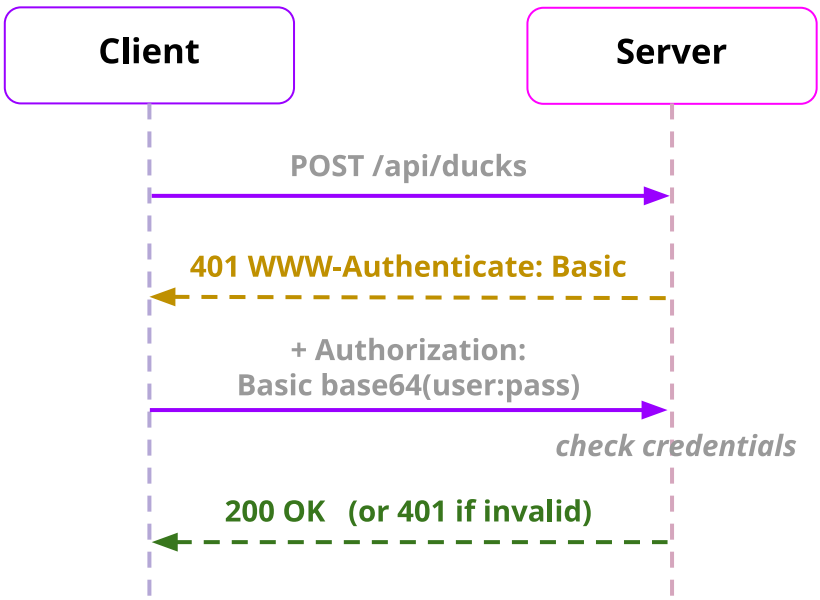
 h2:
   console:
     enabled: true   # http://localhost:8080/
h2-console
```

!! Build a collection → **restart the server** → reload the page: the ducks are still there `ShopState` remembers the mode; `GET /api/ducks/state` restores the view on load

Authentication — the concept

Until now anyone could change the shop. Step one: verify who is calling

- **Authentication** == verifying identity
- Spring Security plugs in as a filter in front of the controllers
- We use **HTTP Basic** — simple, built-in
- Basic is only safe over **HTTPS** (credentials are base64, not encrypted)



Spring Security — users

Define users and **never store plaintext passwords**

```
@Bean
fun userDetailsService(encoder:
PasswordEncoder): UserDetailsService {
   val user =
User.withUsername("user").password(encoder.encode
                  .roles("USER").build()
   val admin =
User.withUsername("admin").password(encoder.encod
                  .roles("ADMIN").build()
   return InMemoryUserDetailsManager(user,
admin)
}

@Bean
fun passwordEncoder(): PasswordEncoder =
BCryptPasswordEncoder()
```

!! *In-memory for the course*: `user` (`USER`) and `admin` (`ADMIN`). Passwords are **BCrypt-hashed** In production this comes from a database or an identity provider

SecurityFilterChain

One place declares who can hit what

```
@Bean
fun securityFilterChain(http: HttpSecurity):
SecurityFilterChain {
   http
       .cors(Customizer.withDefaults()).csrf
{ it.disable() }
       .headers { it.frameOptions { frame →
frame.disable() } }
       .authorizeHttpRequests {
           it.requestMatchers(HttpMethod.PUT,
"/api/ducks").hasRole("ADMIN")
               .requestMatchers(HttpMethod.POST,
"/api/ducks").authenticated()
               /* … other paths … */
               .anyRequest().permitAll()
       }
       .httpBasic(Customizer.withDefaults())
   return http.build()
}
```

!!The chain intercepts every request; matchers map method+path → rule.

Authorization — roles

Authentication asks ***who***; authorization asks ***what*** are you allowed to do

GET permitAll

POST/PATCH/DELETE authenticated

PUT hasRole("ADMIN")

401 Unauthorized

403 Forbidden

Read the shop
anyone — no login

Add / remove / reorder ducks *any logged-in user*

Regenerate the whole shop *admin only*

"who are you?" (missing/bad credentials)

"known, but not allowed"

Authorization — roles

Authentication asks ***who***; authorization asks ***what*** are you allowed to do

```
$ curl -u user:user  -X POST  http://
localhost:8080/api/ducks # 200

$ curl -u user:user  -X PUT  “http://
localhost:8080/api/ducks?mode=List”

# 403  (not ADMIN)

$ curl -X POST  http://localhost:8080/api/
ducks # 401 (no credentials)
```

!!The UI works (frontend sends `admin`)

CORS

The browser blocks cross-origin calls — the server has to opt in

```
@Bean
fun corsConfigurationSource():
CorsConfigurationSource {
    val config = CorsConfiguration().apply {
        allowedOrigins = listOf("http://
localhost:3000") // the dev frontend (npm
start)
        allowedMethods = listOf("GET", "POST",
"PUT", "PATCH", "DELETE", "OPTIONS")
        allowedHeaders = listOf("*")
        allowCredentials = true
    }
    return UrlBasedCorsConfigurationSource()
      .apply { registerCorsConfiguration("/
**", config) }
}
```

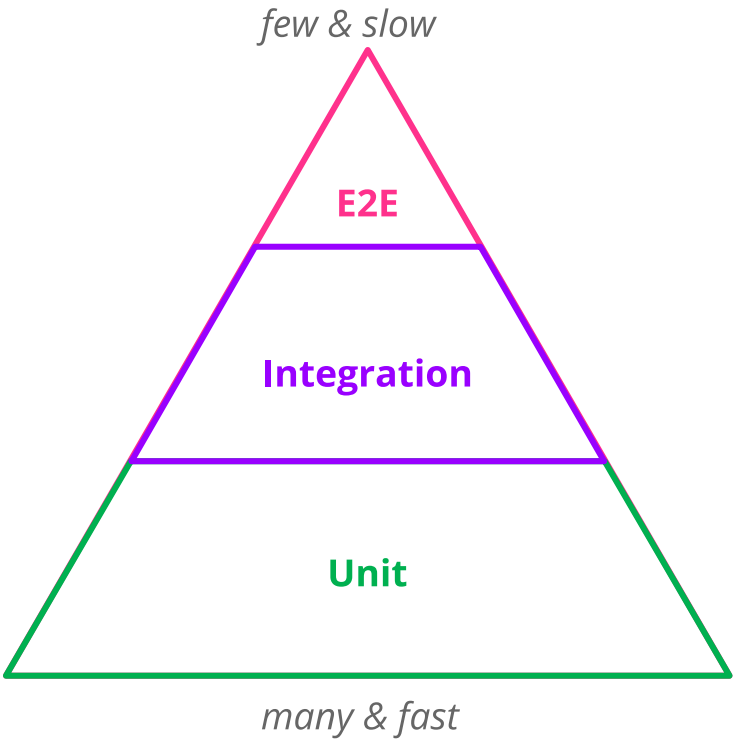
!!Only matters in **dev** (frontend `:3000` → API `:8080`); in prod the jar serves the UI same-origin, so no CORS. `PUT/PATCH/DELETE` trigger a preflight `OPTIONS`

Why tests — the pyramid

We have a secured, DB-backed API — how do we keep it working as we change it?

- Catch regressions before users do
- Document expected behavior
- Make refactoring safe

!!Spring Boot makes all three levels easy: plain JUnit; @SpringBootTest + MockMvc; Testcontainers



Unit tests — pure logic

No Spring context, no database — runs in milliseconds

```
class GameActionFunctionsServiceTest {

   private val service =
GameActionFunctionsService()

   @Test
   fun
`sort puts kotlin ducks first thanks to the
100x coefficient`() = with(service) {
       val plainExpensive = duck("Plain",
kotlin = false, 110) // score 110
       val kotlinCheap = duck("Kot", kotlin =
true, 30)          // score 30 * 100 = 3000
       val sorted = listOf(plainExpensive,
kotlinCheap).sortDucks()
       assertEquals(listOf("Kot", "Plain"),
sorted.map { it.name })
   }
}
```

!!The collection operations (sort/filter/partition/shuffle) are pure functions — the easiest, highest-value tests

Web integration — the setup

Load the real Spring context (including Security) and drive it without a network

```
@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class DuckResourceTest {

    @Autowired
    lateinit var mockMvc: MockMvc

    @Test
    fun `reading the shop is public`() {
        mockMvc.perform(get("/api/
ducks")).andExpect(status().isOk)
    }
}
```

`@SpringBootTest` starts the full context; `MockMvc` drives requests without a real server; `@Transactional` rolls back each test — the seeded catalog stays, the shop changes don't

Testing the security rules

`spring-security-test` injects credentials; we assert the authorization outcomes

```
@Test
fun `mutating without credentials is 401`() {
   mockMvc.perform(post("/api/
ducks")).andExpect(status().isUnauthorized)
}

@Test
fun `replacing the collection requires
ADMIN`() {
   mockMvc.perform(put("/api/
ducks").param("mode",
"List").with(httpBasic("user",
"user")))
       .andExpect(status().isForbidden)   //
403: known, not allowed
}
```

!!See more in `DuckResourceTest`

Real database — the setup

`MockMvc` ran on `H2`. Does it hold on a real PostgreSQL? Testcontainers starts one in Docker

```
@SpringBootTest
@Testcontainers(disabledWithoutDocker = true)
class DuckShopPersistenceTest {

    companion object {
        @Container
        @ServiceConnection
        @JvmStatic
        val postgres =
PostgreSQLContainer("postgres:16-alpine")
    }

    // @Autowired beans + tests …
}
```

!!Testcontainers boots a real PostgreSQL; `@ServiceConnection` auto-points Spring at it — no manual datasource config. `disabledWithoutDocker = true` skips the class cleanly when Docker isn't running

Real database — the relationship

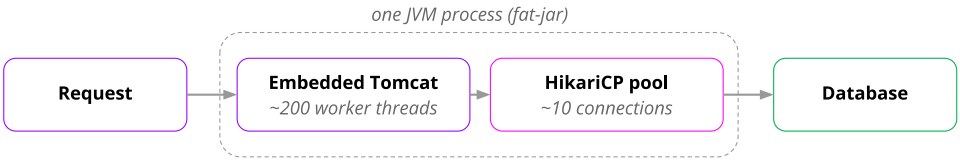
Verify the `Duck` ↔ `Accessory` relationship on the actual database.

```
@Autowired
lateinit var duckRepository: DuckRepository

@Test
fun `the duck-to-accessory relationship is
persisted`() {
    val alex = duckRepository.findAll().first
{ it.name == "Alex" }
    assertEquals(setOf("Monocle", "Hat"),
alex.accessories.map { it.name }.toSet())
}
```

```
$ ./gradlew
test # 13 tests (unit + web on H2);
                  # PG test runs if Docker
is up (else skipped)
```

The backend runtime



!!**Thread-per-request**: a request holds a Tomcat thread; if it waits on the DB and the pool is exhausted, requests queue. *The connection pool is usually the real bottleneck — not CPU*

```
$ curl localhost:8080/actuator/metrics/
hikaricp.connections.active
$ curl localhost:8080/actuator/metrics/
http.server.requests
$ curl localhost:8080/actuator/health  #
{"status":"UP", db: UP}
```

Performance — measure, then choose

Don't guess: **measure** with metrics, find the bottleneck, then pick the method

Measure — metrics
`/actuator/prometheus` exposes JVM, HTTP latency and HikariCP gauges for Grafana/alerts

GraalVM native
`./gradlew nativeCompile` → ahead-of-time binary: startup in ms, low memory

Blocking vs coroutines
Spring MVC = thread-per-request
Kotlin coroutines / WebFlux are non-blocking — fewer threads, more throughput

```
$ curl -s localhost:8080/actuator/prometheus | grep hikaricp
# hikaricp_connections{application="duck-shop-server",pool="HikariPool-1"} 10.0
```

The same shop, a lighter stack

We built duckShop on Spring. Now the same `/api/ducks` — in Ktor

Spring Boot

- A DI container: behaviour comes from annotations + auto-configuration
- Web, Data JPA, Security, Actuator out of the box
- You configure what changes; the framework supplies sensible defaults

Ktor

- No container — you `install()` exactly the features you need
- Routing, serialization, auth are plain Kotlin, written in code
- Lightweight, async by design; fast startup, small footprint

!!**Scope**: We port a subset of the API — GET, PUT, POST on `/api/ducks`. It runs on `:8081`, right next to the Spring app on `:8080`

Setting up Ktor

Dependencies you pick, plugins you install — nothing is implicit

```
plugins {
    kotlin("jvm") version "2.0.21"
    kotlin("plugin.serialization") version
"2.0.21"
    application
}

dependencies {
    implementation("io.ktor:ktor-server-core:
$ktorVersion")
    implementation("io.ktor:ktor-server-netty:
$ktorVersion")
    implementation("io.ktor:ktor-server-
content-negotiation:$ktorVersion")
    implementation("io.ktor:ktor-serialization-
kotlinx-json:$ktorVersion")
    implementation("io.ktor:ktor-server-auth:
$ktorVersion")

implementation("org.jetbrains.exposed:exposed-
core:$exposedVersion")

implementation("org.jetbrains.exposed:exposed-
jdbc:$exposedVersion")
    implementation("com.h2database:h2:2.3.232")
}
```

Setting up Ktor

```
fun main() {
    embeddedServer(Netty, port = 8081) {
module() }
        .start(wait = true)
}

fun Application.module() {
    initDatabase()            // Exposed + H2
file
    configureSerialization() // JSON
    configureSecurity()       // Basic auth
    configureStatusPages()    // uniform errors
    configureRouting(DuckShopService())
}

$ ./gradlew run
INFO  io.ktor.server.Application -
Application started in 0.29 seconds.
INFO  io.ktor.server.Application -
Responding at http://0.0.0.0:8081
```

Routing — annotations vs DSL

```
// Alternative for DuckResource from Spring implementation
fun Application.configureRouting(service: DuckShopService) {
    routing {
        route("/api/ducks") {
            get {
                call.respond(service.all())
            }

authenticate("auth-basic") {
post {

call.respond(HttpStatusCode.Created, service.addRandomDuck()) }
}
}
}
```

}!!Spring discovers handlers from annotations via component scan;

Ktor builds an explicit routing tree in code

Serialization — Jackson vs kotlinx

The same `JsDuck` DTO, two JSON engines

```
implementation("...jackson-module-kotlin")

install(ContentNegotiation) { json() }

// a plain data class —
// Jackson maps it by reflection
data class JsDuck(
     val name: String,
     val description: String?,
     val hasKotlinAttribute: Boolean
)

// compile-time serializer
@Serializable
data class JsDuck(
     val name: String,
     val description: String?,
     val hasKotlinAttribute: Boolean
)
```

!!Jackson works by runtime reflection — no annotations needed; `kotlinx.serialization` generates a compile-time serializer from `@Serializable`

Persistence — Spring Data JPA vs Exposed

Both on file-based H2 (the shop survives restarts) — different styles

```
// Spring: @Entity Duck — here the table IS the entity AND the repository
object DucksTable : Table("ducks") {
   val id = long("id").autoIncrement() // Spring: @Id @GeneratedValue(IDENTITY)
   val name = varchar("name", 100)
   val hasKotlinAttribute = bool("has_kotlin_attribute").default(false)
   override val primaryKey = PrimaryKey(id) // Spring: @Id (declares the PK)
}

transaction { // Spring: @Transactional
     DucksTable.selectAll().map { it.toDuck() } // Spring: DuckRepository.findAll()
}
```

!!JPA hides SQL behind entities & derived-query repositories; Exposed is a thin, typed DSL where you see the query

Security — filter chain vs install + check

Same HTTP Basic, same users — different authorization story

```
install(Authentication) { // Spring: SecurityFilterChain @Bean
    basic("auth-basic") {  // Spring: .httpBasic()
        validate { c → UserPrincipal(c.name, role) } // Spring: UserDetailsService
    }
}

put { // Spring: .requestMatchers(PUT,"/api/ducks").hasRole("ADMIN") — declarative
    val principal = call.principal<UserPrincipal>()
    if (principal?.role ≠ Role.ADMIN) {
        call.respond(HttpStatusCode.Forbidden, ApiError(...))
        return@put
    }
}
```

!!Spring ships rich declarative authorization (`hasRole`, method security); Ktor authenticates and leaves authorization to you — explicit and visible

Security — filter chain vs install + check

!!Try it yourself, and don’t forget to **start** the Ktor server:

```
$ curl -i localhost:8081/api/ducks # 200 public
$ curl -u user:user  -X PUT '.../api/ducks?mode=List' # 403 not ADMIN
$ curl -u admin:admin -X PUT '.../api/ducks?mode=List' # 200 6 duck
```

When Spring, when Ktor?

Same language, same domain, same duckShop:

Spring

- Huge ecosystem: Data, Security, Cloud, Batch, ...
- Auto-configuration & DI — less wiring, strong conventions
- You want sensible defaults out of the box

Ktor

- Coroutine-native, minimal layers, fast startup, small footprint
- You wire exactly what you need — full control, no magic
- Kotlin-first stack (Exposed, kotlinx.serialization)

!!You have seen both frameworks in action with the same app. **The choice is yours.**

JETBRAINS | Kotlin

Thank you

